Final project
Time-series data and application to stock markets

Introduction

Social and economic environment are constantly changing over time, data analysts must be able to assess and predict the effects of these changes, in order to suggest the most appropriate actions to take.

Time-series data is made up by dynamic data collected over time. Thus, it requires to have appropriate forecasting techniques to support business, operations, technology, research, etc.

Objective

The project aims at getting students familiar with time-series data and its applications by analyzing and deriving practical solutions using predictive analytics for stock markets.

This is a free-style project, meaning you have the freedom to decide how to programmatically formulate and solve problems. You'll independently determine the best approach for different tasks, justifying your choices with sound reasoning or experimental results. The evaluation will focus on the accuracy and justification of your solutions.

Project requirements

This project comprises six main tasks, each further broken down into subtasks. The following table provides a detailed overview of the project requirements.

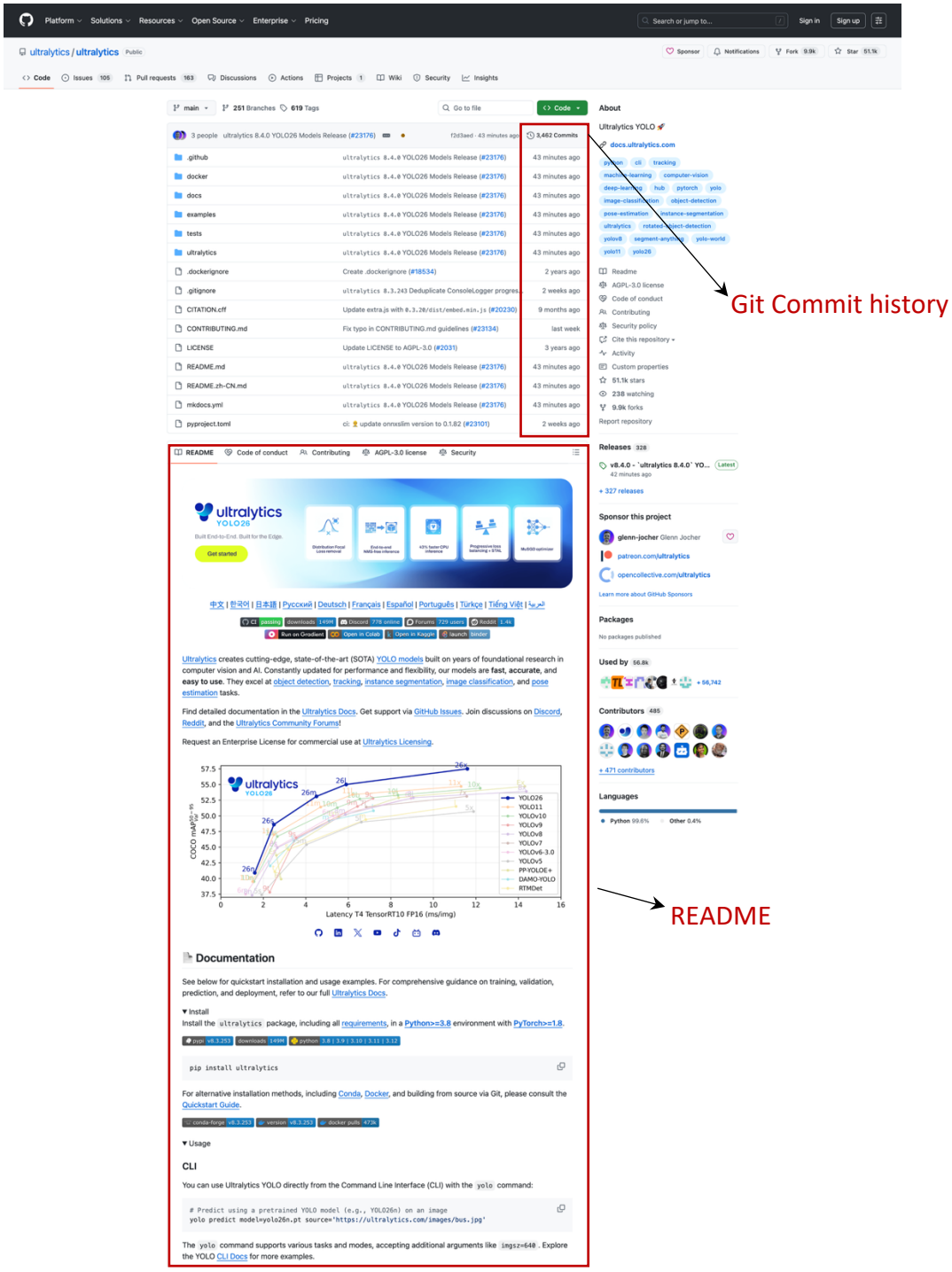
# Task	Name	Requirements
Task 1 (15%)	Nasdaq stock price prediction (Nasdaq dataset)	Nasdaq stock price prediction: • **Task 1.1** (5%, Nasdaq multi-feature extension) Modify the demo code so that the Nasdaq stock price prediction model utilizes multiple features, such as Low, High, Open, Close, Adjusted Close prices, and Volume, rather than relying solely on one feature (Open price). • **Task 1.2** (5%, Nasdaq k^{th} day forecast): Update the demo code to enable the Nasdaq stock price prediction model to forecast the price on the kth day in the future, such as the 3rd day or the 7th day ahead, instead of just the next day. • **Task 1.3** (5%, Nasdaq k days forecast): Extend the demo code so that the Nasdaq stock price prediction model is capable of predicting k *consecutive* days ahead. For example:

		○ If k=3, the model should predict the stock prices for the next three days (i.e., the 1st day, 2nd day, and 3rd day ahead). ○ If k=7, the model should predict the stock prices for the next seven days (i.e., the 1st day, 2nd day, 3rd day, 4th day, 5th day, 6th day, and 7th day ahead). You have to figure out the following points: • Training / Validation / Test split conforming to time-series data. • Cross-validation conforming to time-series data. • Time window, e.g., one-month training and one-week testing. • Company filtering, e.g., those with at least 120 historical data points, companies in certain stock exchanges, companies in certain industries, etc.
Task 2 (15%)	Vietnam stock price prediction (Vietnam dataset)	Vietnam stock price prediction: • **Task 2.1** (5%, Vietnam multi-feature extension): Modify the demo code so that the Vietnam stock price prediction model utilizes multiple features, such as Low, High, Open, Close prices, and Volume, rather than relying solely on one feature (Open price). • **Task 2.2** (5%, Vietnam k^{th} day forecast): Update the demo code to enable the Vietnam stock price prediction model to forecast the price on the kth day in the future, such as the 3rd day or the 7th day ahead, instead of just the next day. • **Task 2.3** (5%, Vietnam k days forecast): Extend the demo code so that the Vietnam stock price prediction model is capable of predicting k *consecutive* days ahead. For example: ○ If k=3, the model should predict the stock prices for the next three days (i.e., the 1st day, 2nd day, and 3rd day ahead). ○ If k=7, the model should predict the stock prices for the next seven days (i.e., the 1st day, 2nd day, 3rd day, 4th day, 5th day, 6th day, and 7th day ahead). You have to figure out the following points: • Training / Validation / Test split conforming to time-series data.

		• Cross-validation conforming to time-series data. • Time window for training and testing. • Company filtering, e.g., those with at least 120 historical data points, companies in certain stock exchanges, companies in certain industries, etc. • Is it good to make use of additional Vietnam data such as dividend history, industry analysis, financial ratio.
Task 3 (20%)	Trading signal identification for Vietnam market	Vietnam trading point prediction: • **Task 3.1** (10%, Buying signal identification): Build a model to identify potential entry points for buying stocks, i.e., the model outputs an indication that can be a score or a probability suggesting it might be a good time to buy. Justify the way you build the model. • **Task 3.2** (10%, Selling signal identification): Build a model to identify potential entry points for selling stocks, i.e., the model outputs an indication that can be a score or a probability suggesting it might be a good time to sell. Justify the way you build the model. You have to figure out the following points: • Training / Validation / test split conforming to time-series data. • Cross-validation conforming to time-series data. • Time window for training and testing. • Company filtering, e.g., those with at least 120 historical data points, companies in certain stock exchanges, companies in certain industries, etc. • Is it good to do the manual feature engineering such as Simple Moving Average (SMA), Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI), etc., to determine the training points?
Task 4 (30%)	Profitable stock selection, risk management and portfolio composition for Vietnam market	Vietnam portfolio and risk management: • **Task 4.1** (10%, Profitable stock selection): Select a list of profitable Vietnamese companies to include in the portfolio. Evaluate the projected profit potential of each chosen company within a designated time frame. Explain the methodology used for portfolio optimization and profit estimation. • **Task 4.2** (10%, Risk management): Identify risky companies that should be excluded from the

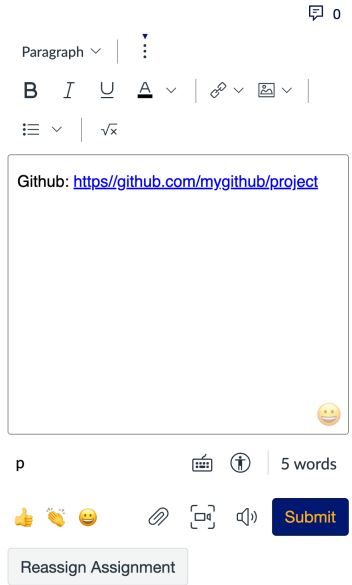
		portfolio. Explain the risk scoring methodology in your risk scoring model. • **Task 4.3** (10%, Portfolio composition): Design a portfolio construction strategy that combines outputs from the profitability and risk models to determine optimal asset allocation. You have to figure out the following points: • Training / Validation split conforming to time-series data. • Cross-validation conforming to time-series data. • Time window for training and testing. • Company filtering, e.g., those with at least 120 historical data points, companies in certain stock exchanges, companies in certain industries. • What should be the list of companies to hold if investors are risk-taking or prudent?
Task 5 (30%, Extra credit)	Industry standard for deployment and ease of use	Industry standard for deployment and ease of use: • **Task 5.1** (5%, Model deployment): Deploy the prediction models as API services. Some keywords to research are Tensorflow Serving (TFServing), REST APIs, gRPC. • **Task 5.2** (10%, Model as SaaS): Deploy the prediction models as a web-based Software-as-a-Service (SaaS). Some keywords to research are TensorflowJS, Superset / Tableau / PowerBI. • **Task 5.3** (15%, AI automation workflow) Design an engineering flow to automate the tasks. Some keywords to research are SQL, MongoDB, Airflow, Airbyte, dbt.
Task 6 (20%)	Report	**Task 6.1** (10%, Report writing) The report: • Should describe your entire journey from Task 1 to Task 5, including your experiments, observations, findings, and conclusions. • Must be at least 2,000 words in length. **Task 6.2** (5%, GitHub Repository) • Must include all code and files for Tasks 1 to 5. • Should be well organized and reproducible. • Commit history should reflect your working process. **Task 6.3** (5%, Repository README)

		• Must describe the project and its tasks. • Must include setup and running instructions. • Should be clear and easy to follow.



Example of a GitHub Repository with a proper README
(https://github.com/ultralytics/ultralytics)

Please make sure to include your GitHub link in the comments section on Canvas:



Data folder structure

- **Nasdaq data**:
  The Nasdaq data folder is organized as follows:
  - ./csv: folder containing historical stock price data.
- **Vietnam data**:
  The Vietnam data folder is organized as follows:
  - ./stock-historical-data: folder containing historical stock price data.
  - ./dividend-history: folder containing historical dividends.
  - ./financial-ratio: folder containing financial health of companies.
  - ./industry-analysis: folder containing analysis of companies in the same industries.
  - companies.csv: list of companies.
  - ticker-overview.csv: overview of companies.

Technology stack

A priori, the project is not limited to any tools, libraries and programming languages. Here follows some examples:

- Main requirements:
  - Python (for programming language).
  - Tensorflow (for deep learning and model serving).
  - Scikit-learn (for machine learning and data analysis).
- Extra credit:
  - SQL/MongoDB (for database).
  - Airflow (for task orchestration).
  - Airbyte (as DB connector).
  - dbt (for data transformation).
  - Superset (for dashboard).

Submission

The structure of submission folder should be organized as follows:

- ./<StudentID>-project-notebook.ipynb: Jupyter notebook containing source code.
- ./<StudentID>-project-report.pdf: project report.

The submission folder is named DL4AI-<StudentID>-project (e.g., DL4AI-2012345-project) and then compressed with the same name.

Evaluation

Project evaluation will be based on successful task completion. Here's a breakdown of the grading criteria:

Task	Subtask	Description	Grade	
Task 1 - Nasdaq stock price prediction (Nasdaq dataset)	Task 1.1	Nasdaq multi-feature extension	5%	15%
	Task 1.2	Nasdaq k^{th} day forecast	5%	
	Task 1.3	Nasdaq k days forecast	5%	
Task 2 - Vietnam stock price prediction (Vietnam dataset)	Task 2.1	Vietnam multi-feature extension	5%	15%
	Task 2.2	Vietnam k^{th} day forecast	5%	
	Task 2.3	Vietnam k days forecast	5%	
Task 3 - Trading signal identification for Vietnam market	Task 3.1	Buying signal identification	10%	20%
	Task 3.2	Selling signal identification	10%	
Task 4 - Portfolio composition, risk management and portfolio optimization for Vietnam market	Task 4.1	Portfolio composition	10%	30%
	Task 4.2	Risk management	10%	
	Task 4.3	Portfolio optimization	10%	
Task 5 (Extra credit) - Industry standard for deployment and ease of use	Task 5.1	Model deployment	5%	30%
	Task 5.2	Model as SaaS	10%	
	Task 5.3	AI engineering workflow	15%	
Task 6 - Report & Github repository setup	Task 6.1	Report writing	10%	20%
	Task 6.2	GitHub repository	5%	
	Task 6.3	Repository README	5%	
Total			**130%**	

Instruction on Extra credit

The extra credit is added because the field of Artificial Intelligence is developing very quickly and new tools, frameworks, and engineering practices appear all the time. To succeed in this environment, students need the ability to search, learn, and experiment with unfamiliar technologies on their own. The purpose of this extra credit is to help students practice researching and self training on current AI technologies, and to understand how these

technologies can be applied in real industry settings. The instructions for completing this extra credit are provided below. These steps are only recommendations, and you are free to adjust them based on the work you have already done in other parts of your project.

Task 5.1 — Model Deployment (5%)

Goal: Deploy your stock price prediction model as an API service so other systems or users can input data (e.g., recent prices, indicators) and receive a predicted value.

Requirements:

- Deploy your trained model as a REST API or gRPC service.
- You may use:
  - TensorFlow Serving (recommended for TensorFlow models)
  - FastAPI / Flask for a custom REST API
  - gRPC (optional, more advanced)

Suggested steps:

- Export the model (e.g., TensorFlow SavedModel or ONNX).
- Run the model server using Docker or a local environment.
- Test using example requests such as:

```
curl -X POST http://localhost:8501/v1/models/stock_model:predict \
-d '{"instances": [[101.5, 102.3, 99.7, 100.8]]}'
```

Documentation (Report) should include:

- How to start the API server
- How to send a prediction request
- Sample input and sample output

Evaluation criteria:

- The API runs without errors
- Prediction results are returned correctly
- Instructions are clear and reproducible

Task 5.2 — Model as SaaS (10%)

Goal: Build a simple web-based service that allows a user to input stock-related data and receive a model prediction, similar to a lightweight financial tool.

Requirements:

- Create a web interface (HTML/JS or React/Vue) that collects user inputs such as:
  - Recent stock prices
  - Technical indicators (depending on how you design your models)
- The interface must call your API (*Task 5.1*).
  - Display the predicted stock price (or trend) clearly.

Suggested steps:

- Use JavaScript (fetch API) to call the backend model service.

- For small models, you may use TensorFlow.js to run inference in the browser.
- Use Superset / Tableau / PowerBI to create mini dashboards using real or predicted stock data.

Documentation (Report) should include:

- How to start the web interface
- How the interface calls the API
- How to input values and read predictions
- Sample screenshots or example inputs and outputs

Evaluation metrics:

- The web interface works end-to-end.
- Predictions display correctly.

Task 5.3 — AI Engineering Workflow (15%)

Goal: Design an automated engineering workflow for stock price prediction, covering data collection, transformation, prediction, and storage.

Requirements:

- Provide a system diagram or clearly written workflow.
- Make use of tools from the list :
  - SQL/MongoDB (https://www.postgresql.org/ / https://www.mongodb.com/)
  - Airflow (https://airflow.apache.org/)
  - Airbyte (https://airbyte.com/)
  - Dbt (https://www.getdbt.com/)
- Explain each step in the pipeline and how the tools interact.

Suggested steps:

- Airbyte: Automatically pull historical and stock market data update (e.g., live stock data).
- dbt: Clean and transform raw price data into feature tables (e.g., Moving Averages, RSI, volatility, etc.).
- Airflow: Schedule the pipeline to run regularly (e.g., hourly or daily):
  - Ingest new stock data
  - Transform using dbt
  - Run your prediction script
  - Store predictions in a database
  - SQL/MongoDB: Store raw, processed, and predicted results for later visualization.
- To deploy a web-based interface, you may consider Python frameworks that are designed for quickly building interactive applications for data science and machine learning models, such as Streamlit or Gradio. **Streamlit** (https://streamlit.io/) is recommended due to its ease of use and developer friendly design. However, the choice of framework is ultimately up to you, depending on which tool you prefer to explore.

Document (Report) should include:

- A clear explanation of the overall goal: designing an automated engineering workflow for stock price prediction that covers data collection, transformation, prediction, and storage
- A system diagram or a clearly written workflow

- A description of how the selected tools are used
- Step-by-step explanations of how data moves through the pipeline

Evaluation metrics:

- Workflow is logical and realistic.
- Tools are used meaningfully.
- Documentation is clear and complete.

Frequently Asked Questions (FAQs)

1. *What modeling approach is required?* A deep learning model must be the core of your solution. Statistical or traditional machine learning methods may be combined with it, but a purely statistical or classical machine learning approach alone is not acceptable.
2. *Do we have to use all the provided data?* No. You are not required to use every dataset if your approach does not need them. In practice, multiple data sources may be available, and part of the task is identifying which ones are truly useful. However, historical price data is mandatory. Additional data may be included if they are logically justified and support your deep learning framework.
3. *Can we download and use additional external data, such as social trend or sentiment data?* Yes. You may collect and incorporate additional external data sources, such as social media trends, sentiment indicators, or other alternative data. This is not required for the project, and you can complete the assignment successfully using the provided data. However, in real world applications, richer and more diverse data often increase the potential for models to learn more informative patterns, provided the data are relevant and properly processed.
4. *How should the data be split?* Chronological order must be strictly preserved. Do not shuffle the data. The training set comes first, followed by validation, and the test set must represent future observations.
5. *Can cross validation be used?* Yes, but only with time series methods such as rolling or expanding windows. Random k fold cross validation is not appropriate.
6. *How should hyperparameters and architecture be selected?* Tune hyperparameters, such as window size, layers, hidden units, batch size, and learning rate, using the validation set. Model design choices must be systematically justified.
7. *Is feature engineering allowed?* Yes. Feature engineering means transforming raw data into informative inputs that improve learning. Examples include moving averages (trend smoothing), volatility indicators (risk measurement), and momentum signals (trend strength). These terms are common in finance and may require background research. Each feature should have a clear rationale.
8. *How should performance be evaluated?* For regression, MAE measures average absolute error, MSE penalizes larger errors more heavily, and RMSE expresses error in the original scale. For classification, accuracy measures overall correctness, precision reflects reliability of positive predictions, recall measures coverage of actual positives, and F1 balances precision and recall. For portfolio strategies, return measures profitability, Sharpe ratio evaluates risk adjusted return, and drawdown measures downside risk.

Choosing appropriate metrics requires domain knowledge and sometimes additional research. Evaluation must respect the chronological split.

9. *Is there a required accuracy threshold?* No fixed target is required. Forecasting accuracy typically decreases as the prediction horizon increases. Assessment focuses on sound methodology and fair validation rather than a specific performance number.
10. *Do we need to predict the entire market?* No. You may focus on companies of interest, especially those with long trading histories, such as over ten years, to ensure sufficient data. Concentrating on a specific sector is also reasonable. Performing well on a carefully selected group is more realistic than modeling the entire market.
11. *What matters most in grading?* Clear problem definition, appropriate deep learning design, careful data selection, proper time series validation, and thoughtful interpretation are more important than achieving the highest numerical result.
12. *How detailed should the report be?* The report should clearly describe your assumptions, preprocessing steps, model design, experiments, evaluation strategy, results, and conclusions. It should reflect your full development process, not only final results.
13. *Is extra credit mandatory?* No. Extra credit is optional. It is intended for students who want to explore industry-level deployment and engineering practices beyond the core modeling tasks.
14. *Can we work in groups?* No. This project must be completed individually to ensure that each student demonstrates independent understanding of deep learning design, data handling, evaluation, and interpretation. The goal is for every student to think critically, make modeling decisions independently, and work through the full pipeline as an AI scientist, from problem formulation to implementation and analysis.
15. *To demonstrate how my code works, besides the report, can I also create a demo, such as a video or an online deployment with a publicly accessible link, especially for the extra credit part?* Yes. In addition to the written report, you are welcome to provide a demonstration, such as a recorded video walkthrough or an online deployment with a public link. This is particularly encouraged for the extra credit component. A clear demo can help illustrate how your system functions in practice, how users interact with it, and how your model performs in a more realistic setting.

Deadline

Please visit Canvas for details.